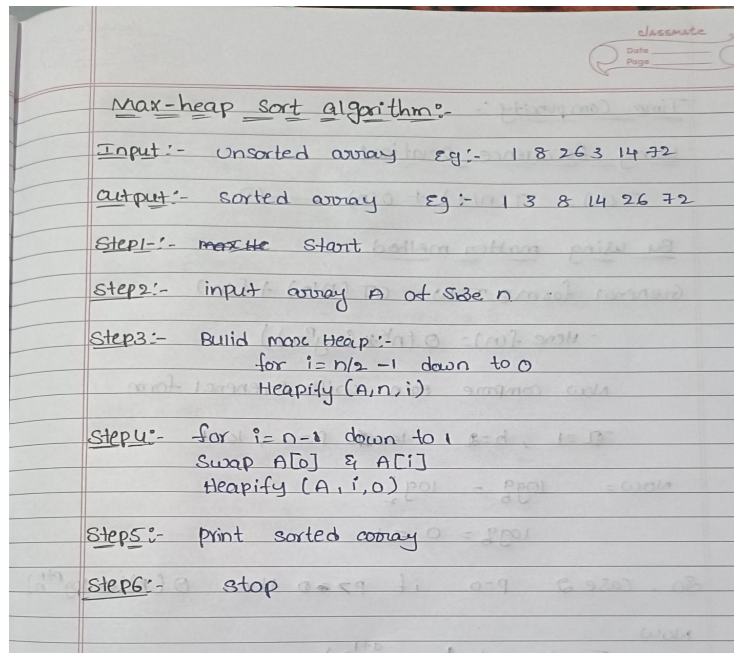


Practical 3

Aim:- To Implementation of max-heap sort algorithm

Alogrithm :-



Code:-

```
#include <iostream>
using namespace std;

// Max Heapify
void heapify(int a[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && a[left] > a[largest])
        largest = left;

    if (right < n && a[right] > a[largest])
        largest = right;

    if (largest != i)
    {
        swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}
```

```

    }
}

// Heap Sort
void heapSort(int a[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);

    // Sorting
    for (int i = n - 1; i > 0; i--)
    {
        swap(a[0], a[i]);
        heapify(a, i, 0);
    }
}

int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int a[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    heapSort(a, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}

```

Output:-

```
C:\Users\manip\OneDrive\Do x + v
Enter number of elements: 5
Enter elements: 26 62 2 6 16
Sorted array: 2 6 16 26 62
Process returned 0 (0x0)   execution time : 14.259 s
Press any key to continue.
```

Time complexity:-

Time Complexity:-

Using Recurrence Relation:-

$$T(n) = T(n/2) + O(n) \quad (1)$$

By using master method

General form $T(n) = aT(n/b) + f(n)$

Here $f(n) = O(n^k \log n)$

Now compare a with general form

$a=1, b=2, k=1$

Now $\log_b a = \log_2 1 = 0 < k$

So, case 2 $p=0$ if $p > 0$ then $O(n^k \log^p n)$

Now

$$T(n) = O(n^1 \log^{0+1} n)$$
$$T(n) = O(n \log n)$$
$$T(n) = O(n \log n)$$

Best Case:-

array is already sorted

eg:- 1 2 3 4 5

$$T(n) = O(n \log n)$$

Average Case:-

array is randomly arranged

eg:- 3 5 6 7 1 2 5

$$T(n) = O(n \log n)$$

Worst Case:-

array is in reverse order

eg:- 4 5 3 5 2 5 1 5 5

$$T(n) = O(n \log n)$$

Conclusion

Max-Heap Sort is an efficient sorting algorithm that uses a **Max Heap** to arrange elements. It sorts the elements in **ascending order** by repeatedly moving the largest element to the end. The time complexity of Heap Sort is **$O(n \log n)$** for the best, average, and worst cases